

Homework 4

by Petros Efraim Koukios, and Alex Rysström
Group 68

February 17, 2026

Problem 8: The Unisex Bathroom problem (40 points)

In this homework, concurrent programming using Java threads and Monitors was used to solve the Unisex Bathroom problem. The problem involves a bathroom, a shared resource, that may be used by either a number of men or women simultaneously, but never by both genders at the same time. The solution must also avoid deadlock and ensure fairness, meaning that no thread might be left waiting indefinitely. This program simulates people working and periodically using the bathroom.

Each thread runs in an infinite loop, alternating between working and using the bathroom. This simulation utilizes the Monitor pattern, a high-level synchronization construct that encapsulates shared data and operations.

Monitor Implementation and Shared State

The solution is encapsulated within a `BathroomMonitor` class. This class maintains the state of the bathroom using four integer counters and a `Turn` variable. Access to these variables is protected using Java's `synchronized` keyword, which ensures that only one thread can execute a critical section of the monitor at a time.

```
class BathroomMonitor {  
    private int menInside = 0;  
    private int womenInside = 0;  
    private int menWaiting = 0;  
    private int womenWaiting = 0;  
  
    private enum Turn { MEN, WOMEN }  
    private Turn turn = Turn.MEN;  
  
    // ... methods ...  
}
```

The `menWaiting` and `womenWaiting` counters are crucial for implementing fairness. They allow the monitor to detect if the opposing gender is waiting, preventing a continuous stream of the current gender from constantly entering.

Synchronization and Entry Logic

Instead of semaphores, this implementation uses Java's `wait()` and `notifyAll()` mechanism. When a thread attempts to enter, it increments the waiting counter and enters a `while` loop to check if it can proceed. The condition for entry is strict to ensure both safety and fairness.

For a man to enter, three conditions must be met:

1. No women are currently inside.

2. No women are waiting while men are inside (preventing starvation).
3. If the room is empty but women are waiting, it must not be the women's specific turn.

```
public synchronized void manEnter(int id) {
    menWaiting++;
    try {
        // wait if women inside, or if women are waiting,
        // or if it is women's turn.
        while (womenInside > 0 ||
            (menInside > 0 && womenWaiting > 0) ||
            (menInside == 0 && womenWaiting > 0 && turn == Turn.WOMEN)) {
            wait();
        }
    } catch (InterruptedException e) { ... }

    menWaiting--;
    menInside++;
    turn = Turn.MEN; // claim turn for men
}
```

This logic creates a "switch" effect. Even if the bathroom is occupied by men, a new man cannot enter if a woman is waiting. This forces the bathroom to eventually empty out, giving the women a chance to enter, ensuring fairness and avoiding starvation.

Exit Logic and Fairness

When a thread exits the bathroom, it decrements the count of people inside. Crucially, the last person to leave checks if anyone from the opposite gender is waiting. If so, they explicitly flip the `turn` variable to the opposite gender before notifying waiting threads.

```
public synchronized void manExit(int id) {
    menInside--;

    // if bathroom is empty and women are waiting,
    // flip the turn to women to prevent race conditions
    if (menInside == 0) {
        if (womenWaiting > 0) {
            turn = Turn.WOMEN;
        }
        notifyAll(); // wake up all threads to recheck conditions
    }
}
```

By updating the `turn` variable before calling `notifyAll()`, we ensure that when the waiting threads wake up, they see the correct state. If a man leaves and sets the turn to women, any men waking up will see the turn is not theirs and go back to sleep, while waiting women will proceed.

In conclusion, the solution shows how Monitors can effectively manage complex synchronization constraints. The use of synchronized methods provides mutual exclusion, while the `wait/notifyAll` pattern manages thread coordination. Fairness is strictly enforced by tracking waiting threads and using a `turn` variable to alternate access when ties occur, guaranteeing that no gender is starved of access to the bathroom.

```

PS C:\Users\petko\Documents\GitHub\Concurrent-Assignments\Assignment4> java UnisexBathroom
- Starting Simulation -
[16:44:52] Woman 4 wants to enter.
[16:44:52] >>> Woman 4 ENTERED. (Women Inside: 1)
[16:44:53] <<< Woman 4 EXITED. (Women Inside: 0)
[16:44:55] Woman 1 wants to enter.
[16:44:55] >>> Woman 1 ENTERED. (Women Inside: 1)
[16:44:56] Woman 3 wants to enter.
[16:44:56] >>> Woman 3 ENTERED. (Women Inside: 2)
[16:44:58] <<< Woman 3 EXITED. (Women Inside: 1)
[16:44:58] Woman 2 wants to enter.
[16:44:58] >>> Woman 2 ENTERED. (Women Inside: 2)
[16:44:59] <<< Woman 1 EXITED. (Women Inside: 1)
[16:45:00] Man 1 wants to enter.
[16:45:02] Man 3 wants to enter.
[16:45:02] <<< Woman 2 EXITED. (Women Inside: 0)
[16:45:02] >>> Man 1 ENTERED. (Men Inside: 1)
[16:45:02] >>> Man 3 ENTERED. (Men Inside: 2)
[16:45:03] Man 2 wants to enter.
[16:45:03] >>> Man 2 ENTERED. (Men Inside: 3)
[16:45:03] Woman 4 wants to enter.
[16:45:04] Man 4 wants to enter.
[16:45:05] <<< Man 1 EXITED. (Men Inside: 2)
[16:45:05] Woman 2 wants to enter.
[16:45:07] <<< Man 2 EXITED. (Men Inside: 1)
[16:45:08] <<< Man 3 EXITED. (Men Inside: 0)
[16:45:08] >>> Woman 4 ENTERED. (Women Inside: 1)

```

Figure 1: Example execution output showing thread activity and synchronization behavior.